

Lab Traps

2026.03.26 张翔

xiangzhang@sjtu.edu.cn 电院群楼3-118

- Backtrace
- Alarm

Backtrace

For debugging it is often useful to have a backtrace: a list of the function calls on the stack above the point at which error occurred

- Implement a `backtrace()` function in `kernel/printf.c`. Insert a call to this function in `sys_sleep` between `#ifdef LAB_TRAPS` and `#endif`, and then run `bttest`, which calls `sys_sleep`. Your output should be a list of return addresses with this form (but the numbers will likely be different):

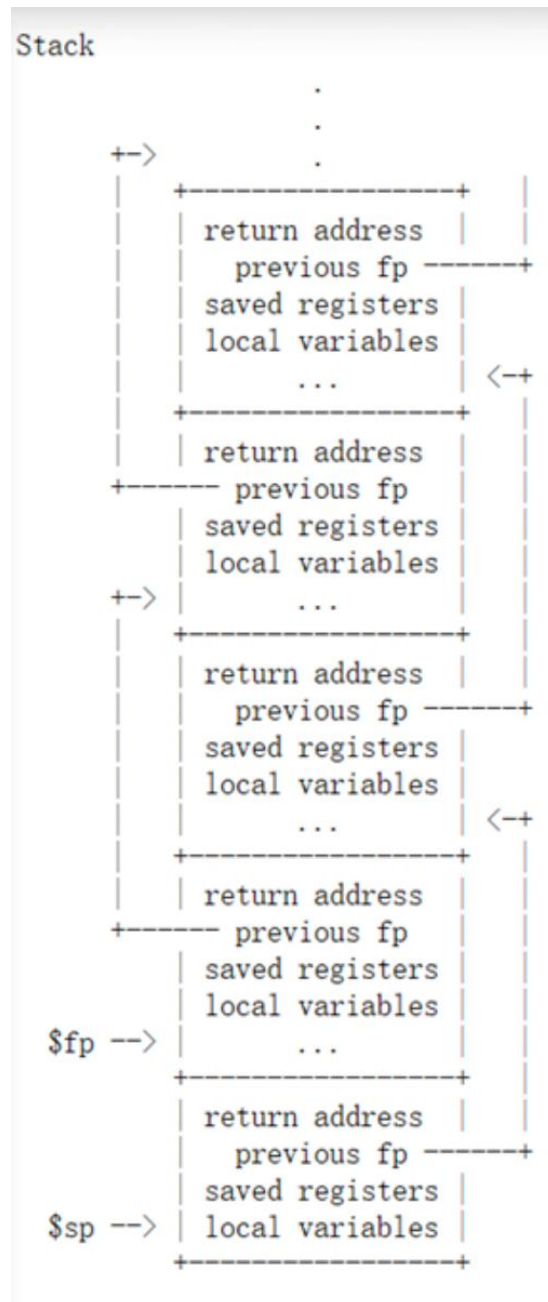
```
backtrace:  
0x0000000080002cda  
0x0000000080002bb6  
0x0000000080002898
```

Backtrace-栈帧

- 函数调用是如何发生的
 - 通过pc跳转、通过ra返回
 - 现场保存
 - 现场恢复
- 栈帧存在的意义：保存函数的私人数据、记录回去的路
- sp (Stack Pointer): 指向当前栈的顶部（图中最低地址处），随入栈出栈动态变化。
- fp (Frame Pointer): 指向当前栈帧的一个固定基准点。
- 关键点：sp 会变，但 fp 与return address的相对位置是固定的。
Fp-8就是return address的地址，fp-16就是这个函数前一个函数fp指针的地址。
- s0 寄存器存放fp指针

Hints

- 使用r_fp()获取fp



地址变小
方向

Backtrace

- 栈帧有多少函数？往上一个一个找返回地址时，何时停止？
 - 在 xv6 中，每个内核栈被分配在一个**对齐的内存页**中。
 - 只要还在同一个内存页中，回溯就是正确的

Hints

- PGROUNDUP(fp)

Backtrace

- 通过addr2line工具转化为对应的函数名

```
$ addr2line -e kernel/kernel  
0x0000000080002de2  
0x0000000080002f4a  
0x0000000080002bfc  
Ctrl-D
```

OUTPUT



```
kernel/sysproc.c:74  
kernel/syscall.c:224  
kernel/trap.c:85
```

- Alarm

Trap

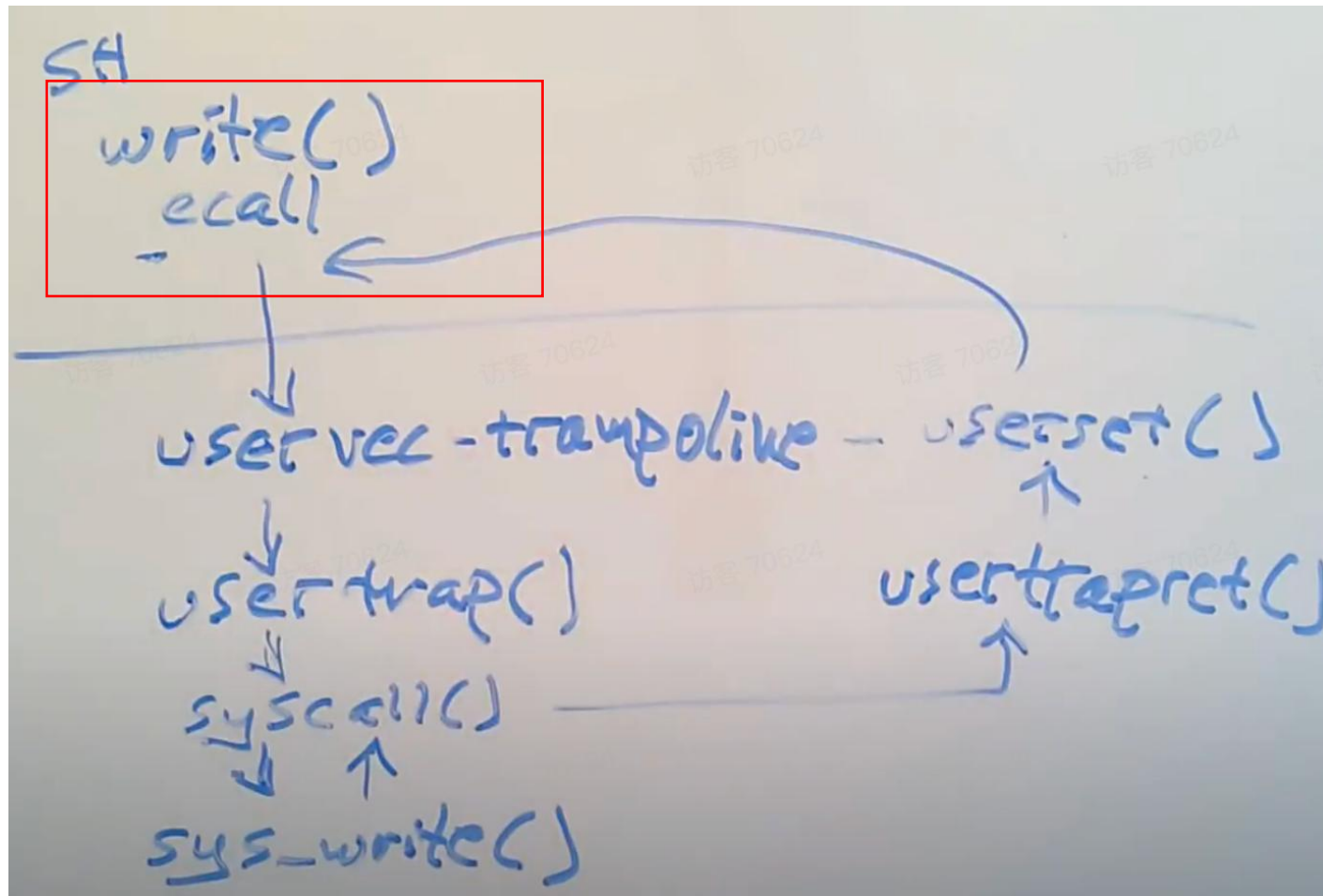
每当：

- 系统调用 (System call): 当用户程序执行 `ecall` 指令，请求内核为其执行某些操作时。
- 异常 (Exception): 指令（无论来自用户还是内核）执行了非法操作，例如除以零或使用了无效的虚拟地址。
- 设备中断 (Device interrupt): 当硬件设备发出信号表示需要处理时，例如磁盘硬件完成了读写请求。

会导致 CPU 暂时搁置普通的指令执行，并强制将控制权转移到处处理该事件的特定代码中。本课程使用trap作为这些情况的通用术语

Trap过程

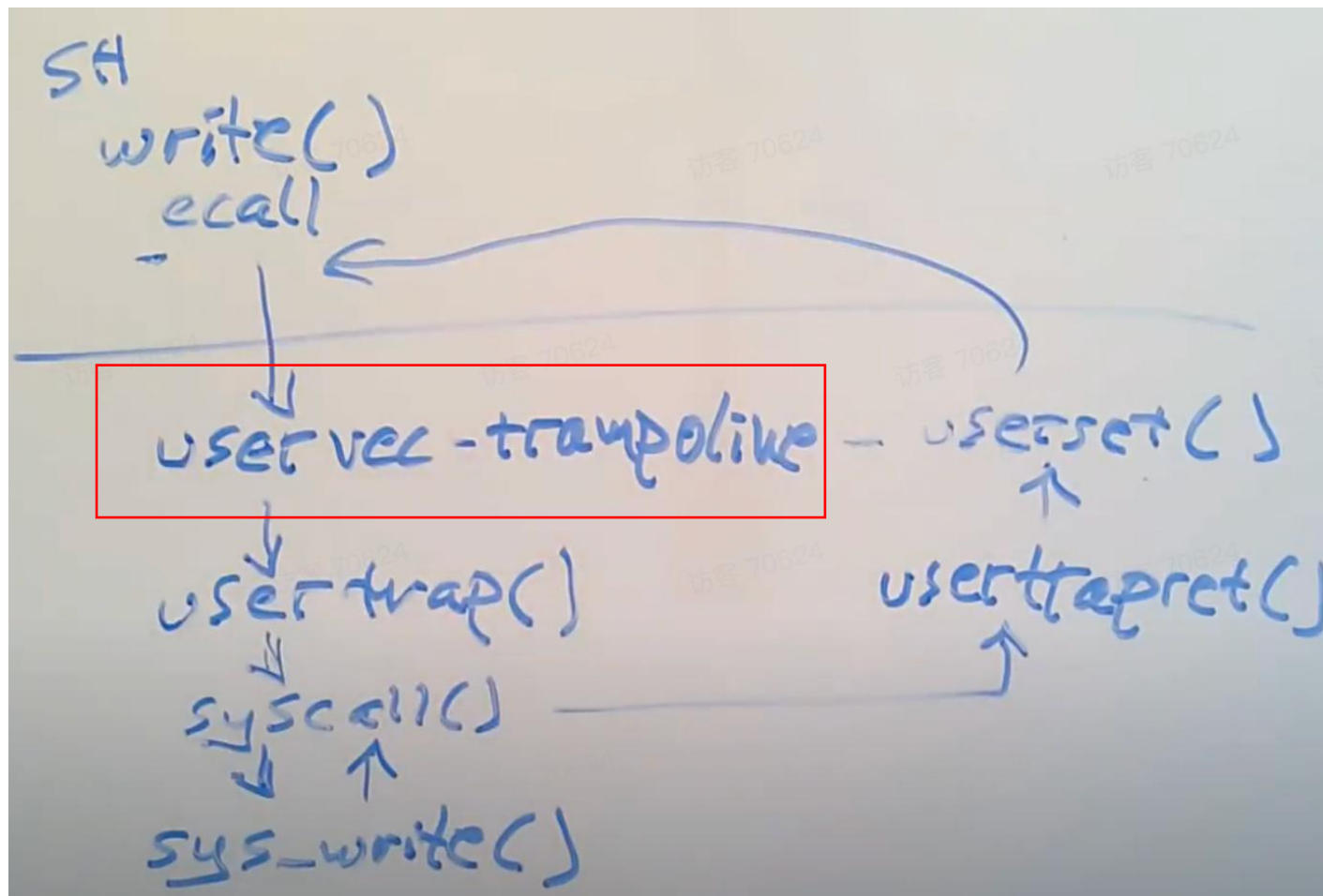
- 以write系统调用为例，讲解执行系统调用时trap过程



Write通过执行ecall指令来执行系统调用。ecall指令会切换到具有supervisor mode的内核中。内核接着第一个执行的指令就是一个由汇编语言编写的函数，叫做uservec，

Trap过程

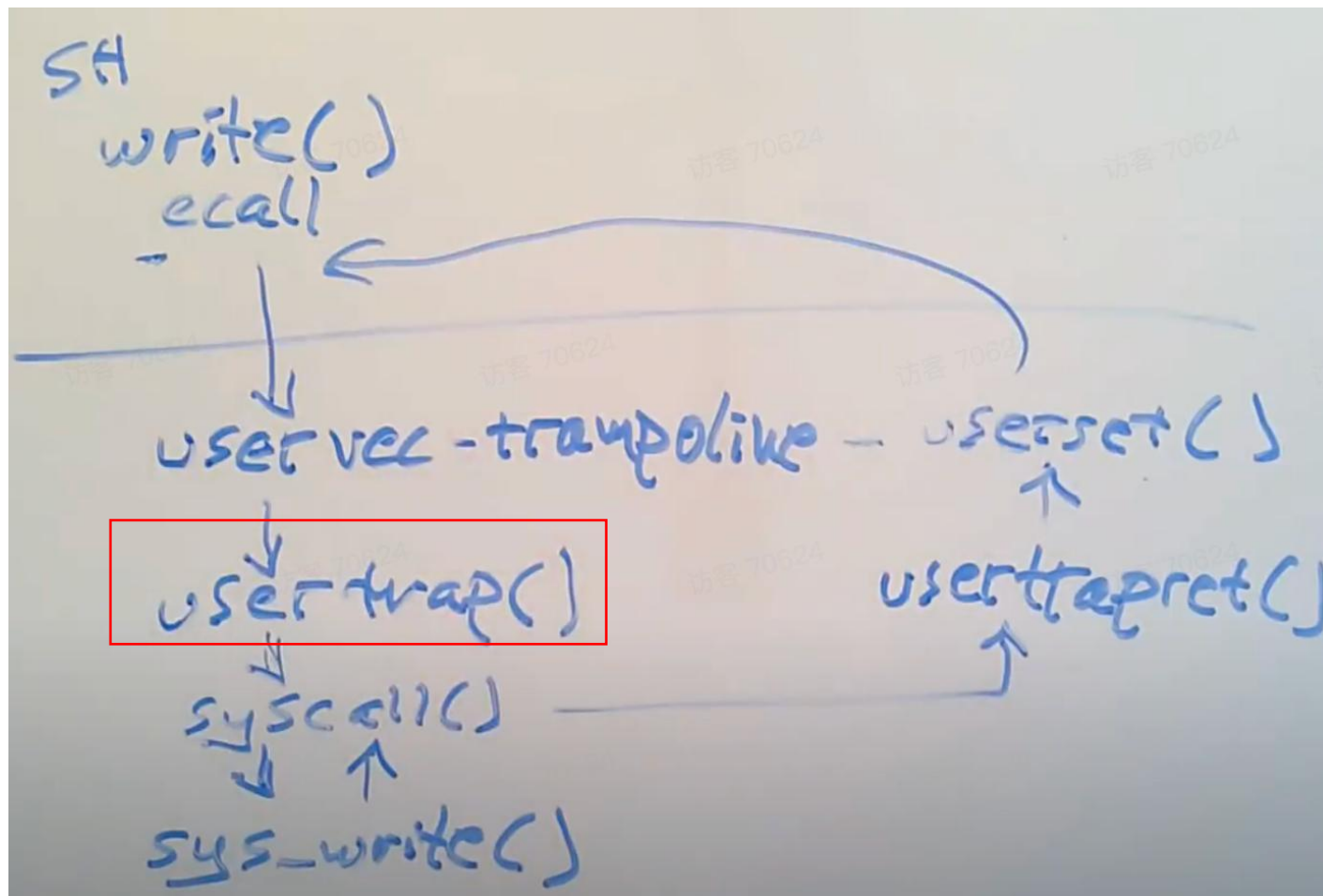
- 以write系统调用为例



uservec这个函数的实现在内核代码trampoline.s文件中。
Uservec函数之后，会跳转到由c语言实现的函数

Trap过程

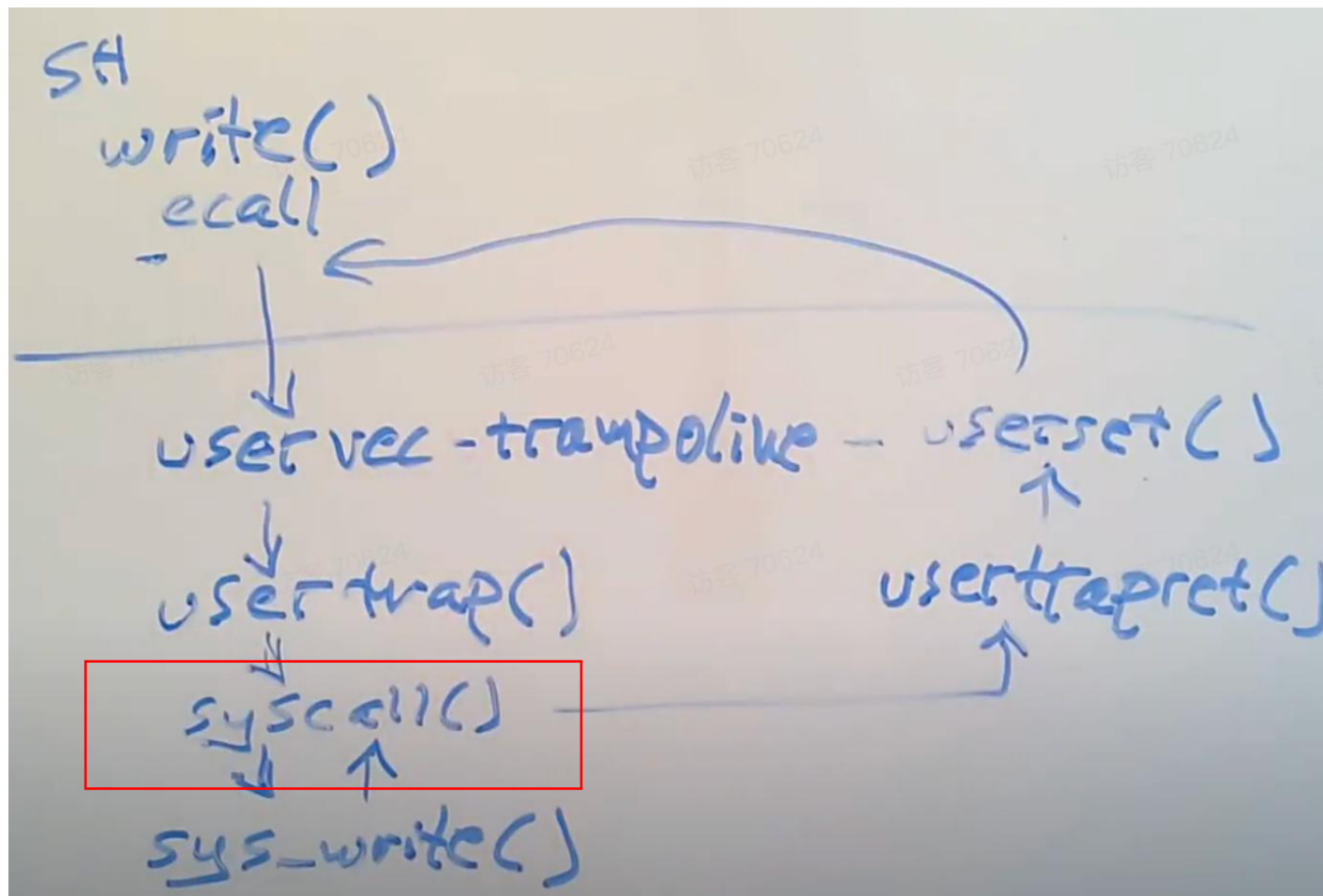
- 以write系统调用为例



usertrap的实现在trap.c中。
在usertrap这个c语言函数中，
执行了一个叫做syscall的函数。

Trap过程

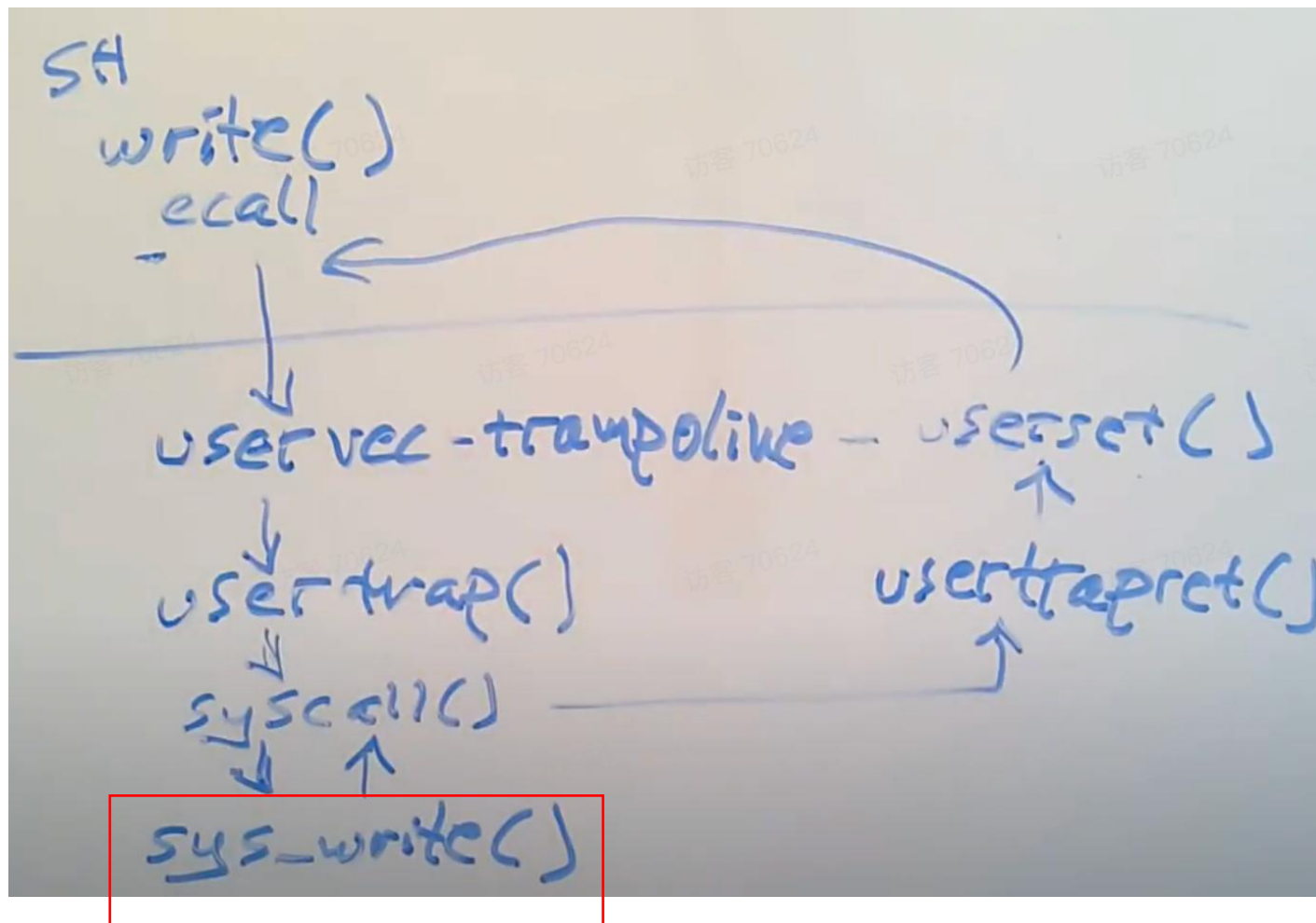
- 以write系统调用为例



syscall这个函数会在一个表中，根据传入的代表系统调用的数字进行查找，并在内核中执行具体实现了系统调用功能的函数。对于我们来说，这个函数就是sys_write。

Trap过程

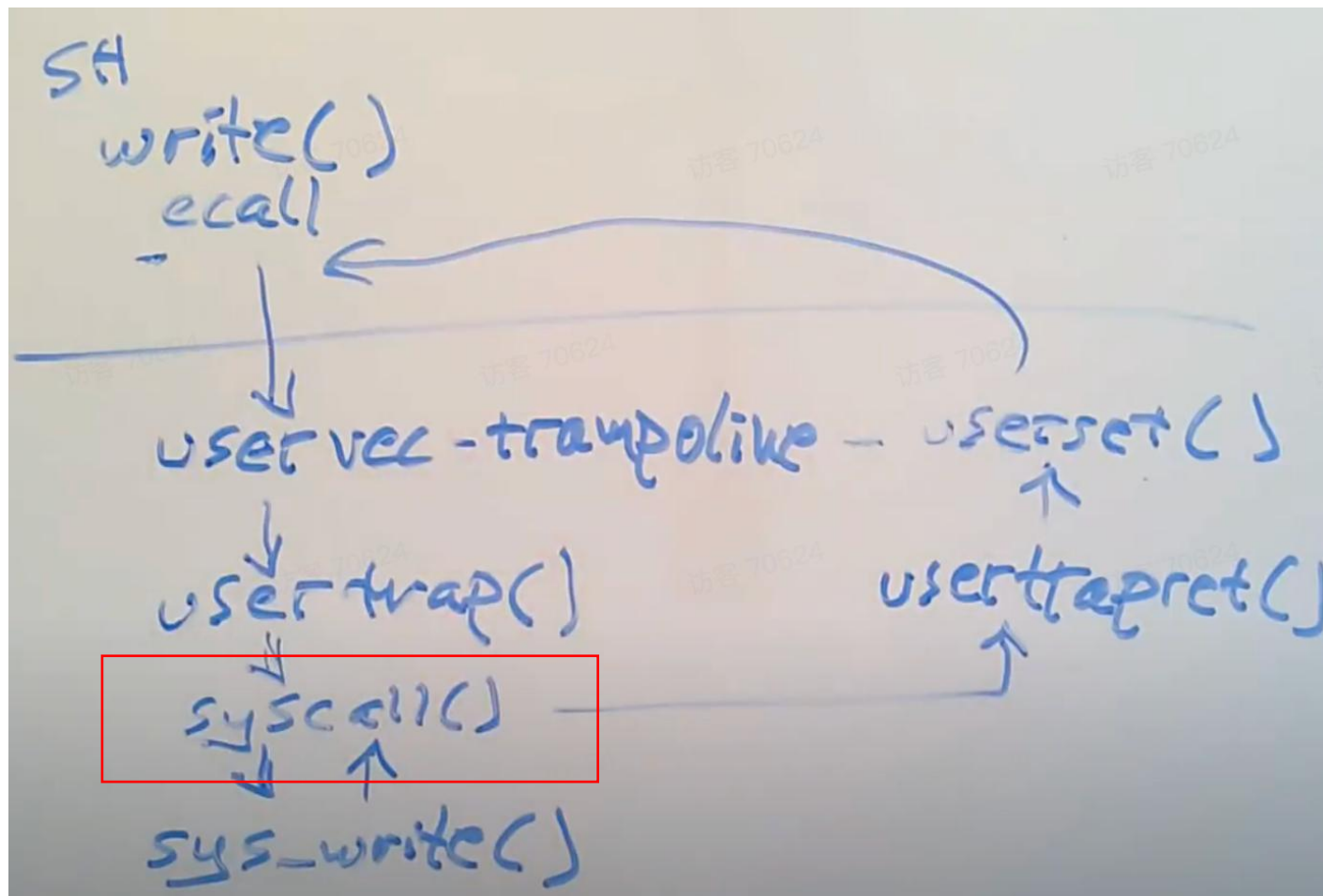
- 以write系统调用为例



`sys_write`会将要显示的数
据输出到console上，当它完
成了之后，它会返回给
`syscall`函数。

Trap过程

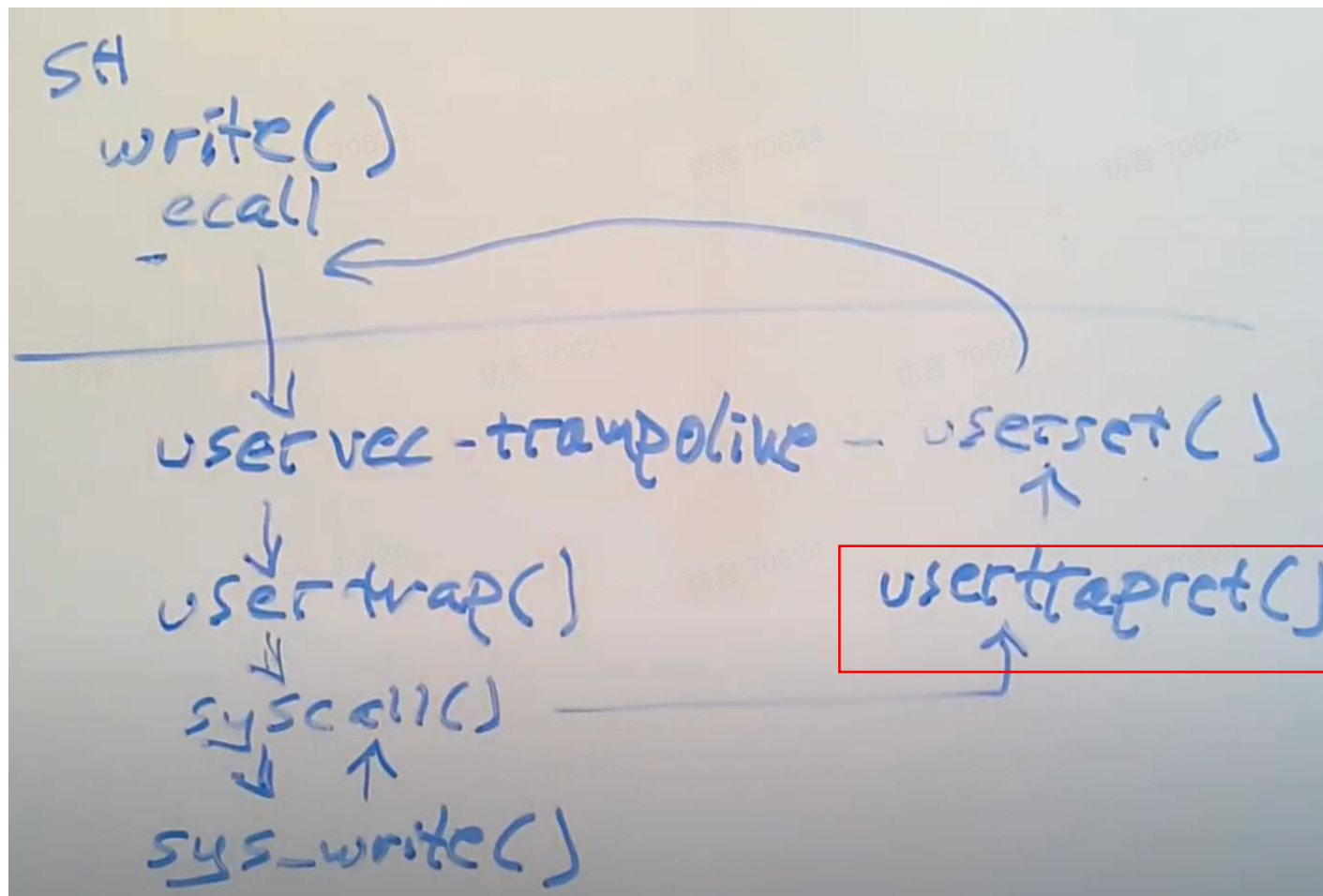
- 以write系统调用为例



因为我们现在相当于在ECALL之后中断了用户代码的执行，为了用户空间的代码恢复执行，需要做一系列的事情。在syscall函数中，会调用一个函数叫做usertrapret，

Trap过程

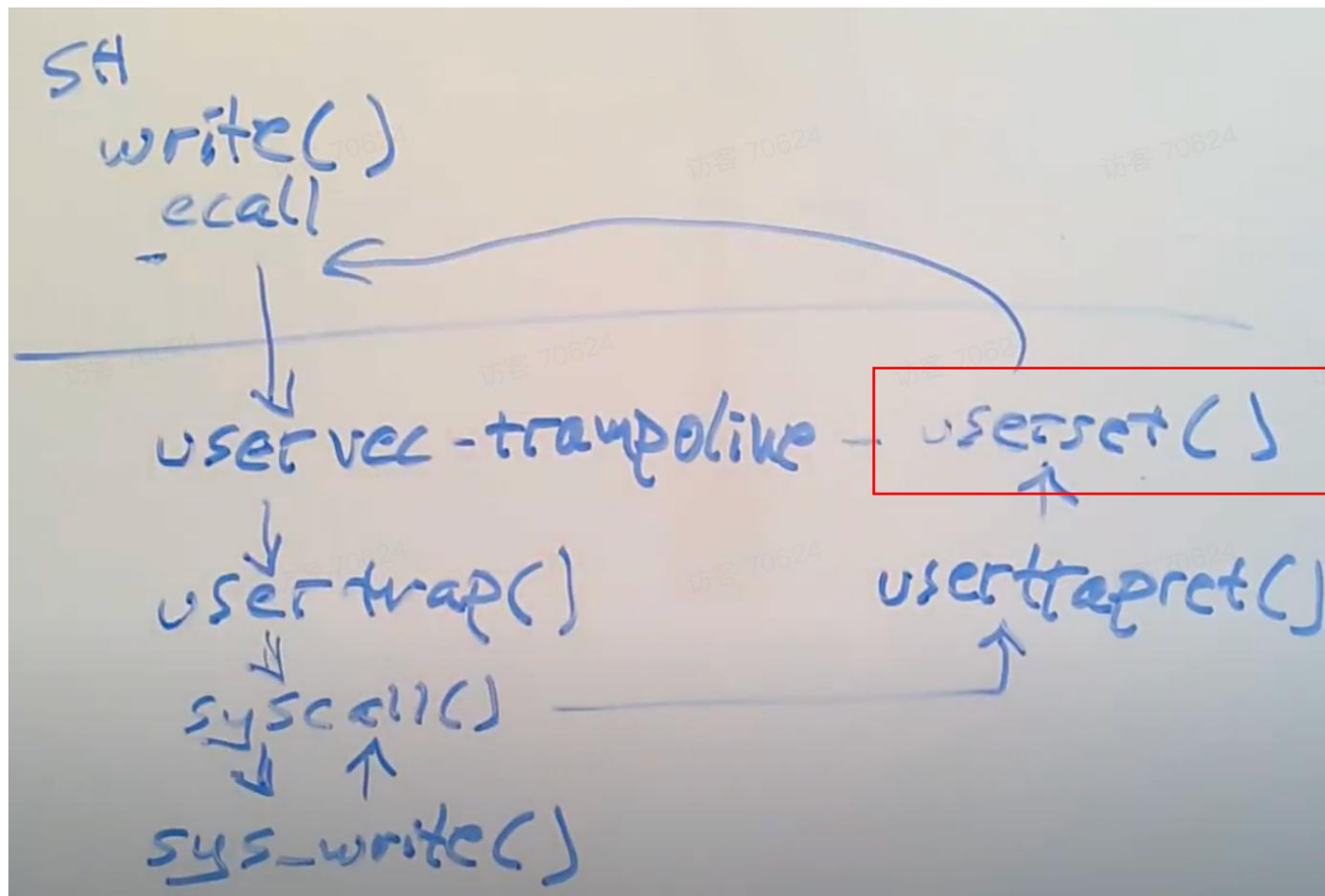
- 以write系统调用为例



`usertrapret`, 它也位于 `trap.c` 中, 这个函数完成了部分方便在C代码中实现的返回到用户空间的工作。

Trap过程

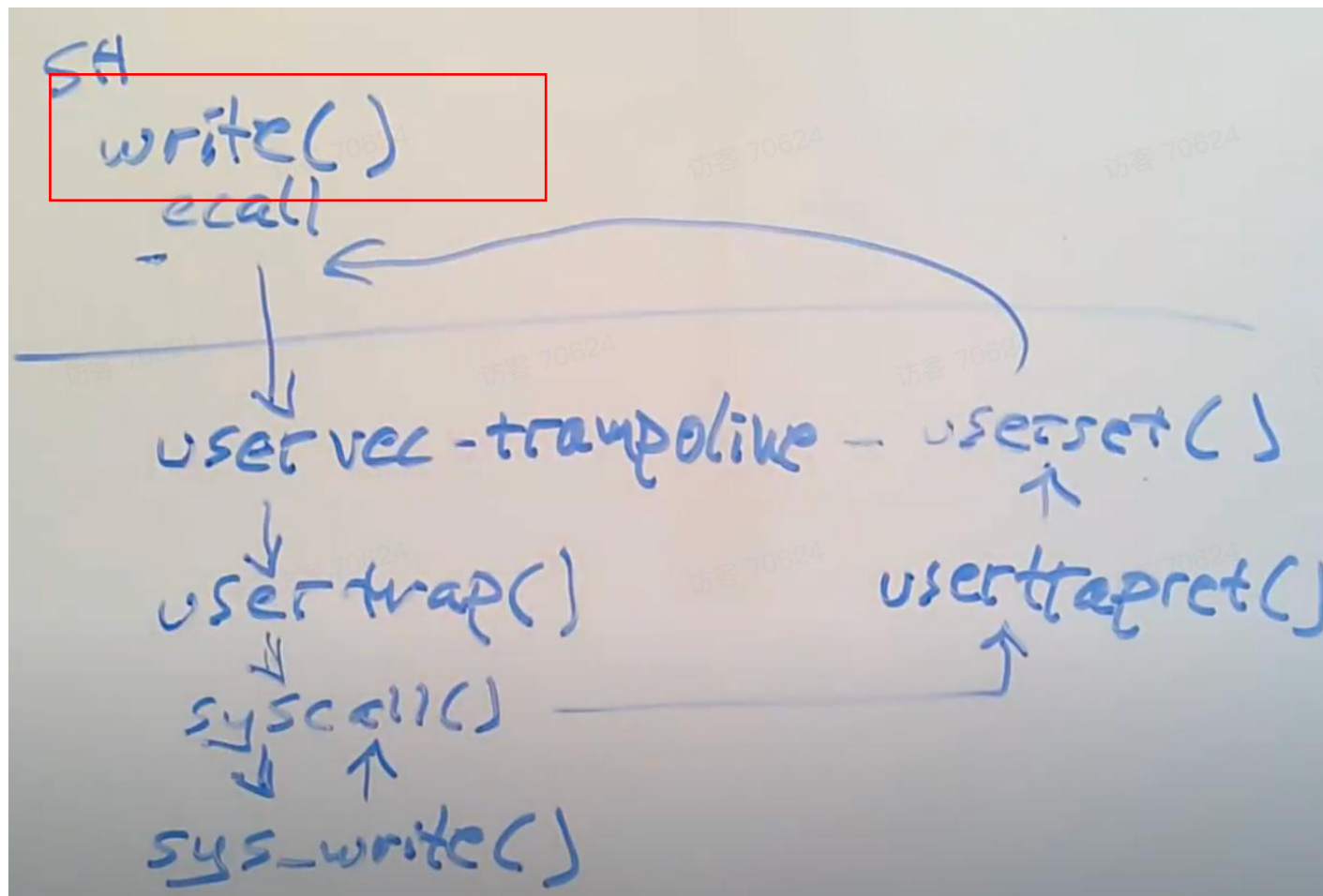
- 以write系统调用为例



除此之外，最终还有一些工作只能在汇编语言中完成。这部分工作通过汇编语言实现，并且存在于 `trampoline.s` 文件中的 `userret` 函数中。

Trap过程

- 以write系统调用为例



最终，在这个汇编函数中会调用机器指令返回到用户空间，并且恢复ECALL之后的用户程序的执行。

Trap过程

- 以系统调用为例

ecall指令

涉及到的寄存器有stvec、sepc、scause、sstatus

执行如下操作

1. 如果陷阱是设备中断，并且状态**SIE**位被清空，则不执行以下任何操作。
2. 清除**SIE**以禁用中断。
3. User mode -> supervisor mode
4. Let sepc = pc
5. Let pc = stvec
6. Jump to pc
7. 设置 `scause` 以反映产生陷阱的原因。
8. 将当前模式（用户或管理）保存在状态的**SPP**位中。

Spec寄存器存放了未来返回的返航坐标

Stvec寄存器存放了trap的进入点，即uservec的地址

Trap过程

- 以系统调用为例

uservec函数

1. 保存现场（32个通用寄存器）
2. 把内核的page table、内核的stack、当前执行该进程的CPU号装载到寄存器里
3. 跳转到usertrap继续执行

Usertrap函数

1. 修改stvec=kernelvec, 保存spec到trapframe
2. 分情况, 执行系统调用/中断/异常的处理逻辑

Usertrapret函数

1. 恢复stvec、sepc的值
2. 保存内核状态到trapframe

Trap过程

- 以系统调用为例

userret函数

1. 恢复现场
2. 把用户空间的page table、用户空间的stack装载到寄存器里
3. 执行sret指令

sret指令

1. 程序会切换回user mode
2. SEPC寄存器的数值会被拷贝到PC寄存器（程序计数器）
3. 重新打开中断

Lab traps: Task2

- 为xv6增加定时器功能
 - 增加两个系统调用: sigalarm 和 sigreturn
 - sigalarm 设置定时器的周期interval和回调函数handler
(sigalarm(interval, handler))
 - 用户态函数funcA调用sigalarm(interval, handler)
 - 每interval个时钟周期, 调用handler一次
 - Sigreturn 在回调函数handler中调用, 恢复被中断的函数状态
 - Handler中调用Sigreturn
 - 返回函数funcA
- Finish task: 4 tests in user/alarmtest.c
 - test0: 实现sigalarm(), 和sigreturn() (返回0), 进入handler中
 - test1: 实现sigreturn(), 在回调函数执行完成后恢复被中断的函数状态
 - test3: handler运行中, 若又过了interval个时钟周期, 不重复调用
 - test4: 恢复原函数funcA的返回值

Step 1: 实现sigalarm(interval, handler)

- 注册系统调用
 - 用户态、内核态
- 你的sys_sigalarm()应该将报警间隔interval和指向处理程序函数的指针handler存储在struct proc的新字段中（位于kernel/proc.h）。
- 你也需要在struct proc新增一个新字段。用于跟踪自上一次调用（或直到下一次调用）到进程的报警处理程序间经历了多少滴答；可以在proc.c的allocproc()中初始化proc字段。
- 每过一个时钟周期，硬件时钟就会强制一个中断，这个中断在kernel/trap.c中的usertrap()中处理。时钟中断的处理具体是在usertrap中 if (which_dev == 2) 分支
- 需要修改usertrap()，以便当进程的报警间隔期满时，用户进程执行handler函数。
 - 如何返回到用户态的handler？（返回到用户态的地址在哪？）
- make CPUS=1 qemu-gdb：指定单核，便于调试

Step 2: 实现sigreturn()

- handler函数结束后会调用sigreturn(), 返回到被中断函数funcA
 - user(funcA)->kernel->user(handler)->kernel->user(funcA)
- 如何保存被中断函数funcA的上下文? (struct proc)
- 防止对handler程序的重复调用——如果正在执行handler程序, 即使到了interval的时间, 内核也不应该再次调用它。(e.g. 标记handler_executing)
- 不要忘记恢复funcA返回值 (a0)

Optional Challenge

- Print the names of the functions and line numbers in `backtrace()` instead of numerical addresses

Lab

- lab-traps-report.txt
 - 描述一下自己做lab的时候遇到的问题以及解决方法
 - 或者一些帮助自己写lab的小技巧
 - 中英皆可
- 更详细的介绍
 - MIT xv6 Book Chapter 4(canvas)
 - gdb_slides.pdf (canvas)
 - Lab2 traps.pdf(canvas)
 - MIT 6.1810 课程笔记
 - <https://mit-public-courses-cn-translatio.gitbook.io/mit6-s081/lec06-isolation-and-system-call-entry-exit-robert/6.5-uservec>
- 提交
 - 见lab文档

评分

- 评分由以下几部分构成
 - 基本测试通过（make grade 通过测试） 2分
 - 代码逻辑功能正确 2分
 - 报告 5分
 - 注释和代码可读性 1分

Q&A

- 任何问题?

END